Computer Network
Assignment – 1

1. Explain the functions of the Data Link Layer.

Ans: The Data Link Layer is the second layer in the OSI model. Its main responsibility is to ensure the reliable and error-free transfer of data frames between two connected nodes in a physical link.Functions of data link layer are:

1. Framing

The network layer sends data in packets. The data link layer receives these packets and divides them into manageable units called frames. It then adds a header and a trailer to each frame.

- Header: Contains control information, such as the source and destination of MAC addresses.
- Trailer: Typically contains error-checking information.

2. Physical Addressing

The data link layer adds physical addresses (MAC addresses) to the frame header. This ensures that the frame reaches the correct device on the local network.

- Source MAC: The address of the sender.
- Destination MAC: The address of the intended receiver.

3. Error Control

Since physical transmission media are prone to errors (due to noise, interference, etc.), the data link layer implements mechanisms to detect and sometimes correct these errors.

- Error Detection: It adds a sequence of bits, often a Cyclic Redundancy Check (CRC), to the trailer of the frame. The receiver calculates the CRC and compares it to the one in the trailer. If they don't match, an error occurs.
- Error Correction: In some cases, the layer can request the retransmission of corrupted frames (Automatic Repeat reQuest - ARQ).

4. Flow Control

If a sender is transmitting data faster than the receiver can process it, the receiver's buffers will overflow, leading to data loss. Flow control prevents this by regulating the amount of data sent before requiring an acknowledgement. Common mechanisms include:

- Stop-and-Wait: The sender transmits one frame and waits for an acknowledgment before sending the next.
- Sliding Window: Allows multiple frames to be transit before requiring an acknowledgment, improving efficiency.

5. Access Control (Medium Access Control - MAC)

When multiple devices share a single physical communication medium (like an Ethernet hub or a Wi-Fi channel), there needs to be a set of rules to determine which device gets to transmit at any given time. This prevents collisions (when two devices send data simultaneously, corrupting both signals.

- CSMA/CD (Carrier Sense Multiple Access with Collision Detection): Used in traditional wired Ethernet.
- CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance): Used in wireless networks (Wi-Fi).

2. What is framing?

Ans: Framing is a crucial function of the Data Link Layer. It is the process of taking the data packet received from the Network Layer and encapsulating it into a structured, manageable unit called a frame before it is transmitted as raw bits over the physical network.
You can think of framing like taking a letter (the data packet) and putting it into an envelope (the frame), then writing the physical sender and receiver addresses on the outside so the postal service knows exactly where to deliver it locally.

The Structure of a Frame

While different networking protocols (like Ethernet or Wi-Fi) have slightly different frame formats, a standard frame generally consists of three main parts:

1. Header: This is attached to the front of the data and contains control and addressing information. It typically includes:
    a. Destination MAC Address: The physical address of the device meant to receive the frame.
    b. Source MAC Address: The physical address of the device sending the frame.
    c. Control Information: Details about the protocol type, frame length, or synchronization bits to alert the receiver that a frame is arriving.
2. Payload (Data Field): This is the core "cargo" being transported. It contains the actual packet handed down from the layer above (the Network layer).
3. Trailer: This is appended to the end of the frame. Its primary purpose is error detection. It almost always contains a Frame Check Sequence (FCS), which is a mathematical value (like a CRC) calculated based on the frame's contents. The receiver recalculates this value; if it doesn't match the trailer, the frame is discarded as corrupted.

Why is Framing Necessary?

When data travels across the physical layer (like a copper cable or radio waves), it is simply a continuous, unbroken stream of electrical or optical signals representing 1s and 0s.

Framing is necessary because it imposes order on this chaos. It provides:

- Message Boundaries (Synchronization): The receiver needs to know exactly where one message ends, and the next begins. Framing defines this clear start and stop points within the continuous stream of bits.
- Granular Error Checking: By breaking a massive stream of data into smaller frames, error checking becomes efficient. If a single bit is flipped by electrical noise, only the affected frame needs to be retransmitted, rather than the entire data stream.
- Physical Routing: The framing process applies to the MAC addresses necessary for switches to direct the data to the correct physical port on a local network.

How Does the Receiver Find the Frame?

Because the data stream is continuous, the Data Link layer must use specific techniques to mark the start and end boundaries of a frame. Two common methods include:

- Bit Stuffing: A specific, unique sequence of bits (for example, 01111110) is used as a "flag" to mark the beginning and end of a frame. To prevent the receiver from getting confused if the *actual data* happens to contain that exact sequence, the sender's hardware automatically inserts (stuffs) an extra 0 whenever it sees five consecutive 1s in the data. The receiver then strips that extra 0 out.
- Byte Stuffing: Similar to bit stuffing, but it uses specific 8-bit characters (bytes) as flags. If the flag character happens to appear naturally within the payload data, an "escape" character is inserted right before it so the receiver knows it is part of the data and not the end of the frame.

3. Explain error detection techniques.

Ans: When data travels over physical network cables or wireless signals, it is susceptible to electrical noise, interference, and signal degradation. These factors can easily flip a 0 to a 1 or vice versa, corrupting the message.

Error detection techniques are mathematical methods used by the receiving device to determine if the data it received is an exact, uncorrupted copy of what was sent.

1. Parity Check

This is the simplest, most basic form of error detection. It involves adding a single extra bit, called a parity bit, to a small block of data (usually a 7-bit ASCII character) before transmission.

- Even Parity: The sender calculates the number of 1s in the data. The parity bit is then set to either 0 or 1 to ensure that the *total* number of 1s (data + parity bit) is an even number.
- Odd Parity: The parity bit is set to ensure the total number of 1s is an odd number.

When the receiver gets the data, it counts the 1s. If even parity was used, but the receiver counts an odd number of 1s; it knows a bit was flipped during transit and an error occurred.

Limitation: A simple parity check can only detect an odd number of errors (1, 3, 5 flipped bits). If exactly two bits flip in the same block, the total number of 1s remains even (or odd); the parity check passes, and the error sneaks through undetected.

Two-Dimensional Parity Check:

To solve the limitation of simple parity, data is arranged in a two-dimensional grid (a block). Parity bits are calculated for every row *and* every column. This "grid" approach greatly increases the likelihood of catching burst errors (multiple flipped bits) and can even help pinpoint exactly which bit was corrupted.

2. Checksum

The checksum technique is commonly used in the Network and Transport layers (like in TCP/IP). Instead of looking at individual bits, it treats segments of data as binary numbers and uses addition.

1. The Sender: The data is divided into equal-sized segments (typically 16-bit blocks). The sender adds all these segments together using 1 complement arithmetic. The final sum is then complemented (every 0 becomes a 1, and every 1 becomes a 0). This resulting value is the checksum, which is attached to the end of the packet.
2. The receiver divides the incoming data into the same 16-bit segments and adds them all together, *including* the checksum value that was sent.
3. The Check: If no errors occurred, the addition of the data and its inverted checksum will result in a final sum of all 1s. If there is even a single 0 in the result, the packet is corrupted and discarded.

3. Cyclic Redundancy Check (CRC)

CRC is the most robust, complex, and widely used error detection method at the Data Link Layer (it's what sits inside the trailer of an Ethernet or Wi-Fi frame). It is based on binary division using polynomials.

1. The Setup: Both the sender and receiver agree on a specific binary sequence called a generator polynomial (the divisor).
2. The Sender's Math: The sender appends a string of 0s to the end of the data. It then performs modulo-2 division (which relies on XOR logic gates rather than standard subtraction) on this new data string, dividing it by the generator polynomial.
3. The Remainder: The remainder of this complex division is the actual CRC value. The appended 0s are replaced with this CRC value, and the frame is transmitted.
4. The Receiver's Math: The receiver takes the entire incoming frame (the data plus the CRC value) and divides it by that exact same generator polynomial using modulo-2 division.
5. The Check: If the remainder of the receiver division is exactly zero, the frame is flawless. Any non-zero remainder proves the frame was corrupted.

Because of the complex division involved, CRC is incredibly effective at catching single-bit errors, odd numbers of errors, and massive burst errors where large chunks of data are corrupted at once.

4. Explain parity check.

Ans: A parity check is the simplest method used in networking to detect errors that might occur when

data is transmitted over a physical medium. It works by adding a single extra bit—called a parity bit—to a specific block of data (usually a 7-bit or 8-bit character) before it is sent.
The sole purpose of this extra bit is to make the total number of 1s in that block of data either an even number or an odd number, depending on the agreed-upon rule between the sender and receiver.

Here is a breakdown of the two types of parity checks and exactly how the math works.

1. Even Parity

The rule here is that the total number of 1s in the transmitted data (including the parity bit itself) must always be an even number.

2. Odd Parity

The rule here is that the total number of 1s in the transmitted data (including the parity bit) must always be an odd number.

Step-by-Step Example: Sending the letter "C"

Let's say a computer wants to send the 7-bit binary code for the uppercase letter "C", which is 1000011.

If the network uses Even Parity:

- Count the 1s: The sending device looks at the data 1000011 and counts exactly three 1s.
- Set the Parity Bit: Because three is an odd number, the sender must add a 1 to the end of the data to make the total number of 1s even (four is an even number).
- Transmit: The final 8-bit data block sent over the network cable is 10000111.

If the network uses Odd Parity:

- Count the 1s: The sending device looks at 1000011 and counts three 1s.
- Set the Parity Bit: Because three is already an odd number, the sender adds a 0 to the end of the data to keep the total number of 1s odd.
- Transmit: The final 8-bit data block sent over the network cable is 10000110.

How the Receiver Catches the Error

Let's assume the computers are using Even Parity, and the sender transmits our data block: 10000111.

- Perfect Transmission: The receiving computer gets 10000111. It counts the 1s and finds four. Four is an even number. The parity check passes, and the data is processed.
- The Corrupted Transmission: While traveling over the wire, electrical interference flips the second bit from a 0 to a 1. The receiving computer actually gets 11000111. It counts the 1s and finds five. Because five is an odd number, but the network rule is Even Parity, the receiver instantly knows an error occurred and discards the data.

The Major Limitation of Parity Checks

While parity checks are fast and require very little processing power, they have one significant flaw: they can only detect an odd number of errors (if 1, 3, or 5 bits are flipped).

If exactly two bits are flipped by interference (an even number of errors), the parity check will fail to catch it. For example, if our original 10000111 (Even Parity) gets corrupted into 11100111, the receiver will count six 1s. Six is an even number, so the receiver will falsely assume the data is perfectly fine. This is why more complex methods like Checksums and CRCs are used for critical network traffic.

5. Explain CRC (Cyclic Redundancy Check).

Ans: The Cyclic Redundancy Check (CRC) is the most powerful and widely used error detection technique at the Data Link Layer. You will find it inside the trailer of almost every Ethernet frame and Wi-Fi packet. While parity checks only look at the number of 1s, CRC treats the entire block of data as one giant binary number and performs a specific type of mathematical division on it.

Here is how the CRC process works conceptually and mathematically:

The Core Concept: Modulo-2 Arithmetic (XOR)

CRC relies on binary division, but instead of standard subtraction used in normal long division, it uses Modulo-2 arithmetic.

Modulo-2 addition and subtraction are identical, and both are performed using the XOR (Exclusive OR) logic gate. The rule for XOR is simple:

- If the bits are the same (0 and 0, or 1 and 1), the result is 0.
- If the bits are different (0 and 1, or 1 and 0), the result is 1.

The Setup: The Generator Polynomial

Before any communication begins, both the sender and the receiver must agree on a specific binary divisor, commonly referred to as the Generator of Polynomial.

This generator is usually standardized depending on the network protocol (e.g., Ethernet uses a specific 33-bit generator called CRC-32).

The Sender's Process (Creating the CRC)

When the sender has a block of data ready to transmit, it follows these steps:

1. Append Zeros: The sender takes the original data and appends a string of 0s to the end of it. The number of 0s added is exactly one less than the length of the generator polynomial. (For example, if the generator is 4 bits long, the sender appends three 0s).
2. Binary Division: The sender takes this newly expanded data and divides it by the agreed-upon generator polynomial using Modulo-2 (XOR) division.
3. The Remainder: In this division, we don't care about the quotient (the answer at the top). We only care about the remainder at the very bottom. This remainder *is* the CRC value (also known as the Frame Check Sequence or FCS).
4. Transmit: The sender replaces the appended 0s with the calculated CRC value and transmits the entire frame across the network.

The Receiver's Process (Checking the Data)

When the receiver gets the frame (which now contains the original data plus the CRC value at the end), it performs a very similar mathematical check:

1. Divide Again: The receiver takes the entire incoming frame and divides it by the exact same generator polynomial using the Modulo-2 (XOR) division.
2. Check the Remainder: * If the remainder of the receiver division is exactly zero, it mathematically proves that the data arrived perfectly intact. The data is accepted and passed up to the Network layer.
    a. If the remainder is anything other than zero, it means one or more bits were flipped during transit. The receiver immediately discards the frame.

Why is CRC so Popular?

CRC requires more complex hardware to calculate quickly compared to a parity check, but it is incredibly robust. It is highly effective at detecting:

- Single-bit errors.
- Odd numbers of errors.
- Burst errors: This is where CRC shines. If a sudden spike of electrical noise wipes out a long, contiguous sequence of bits (a burst), simple parity or checksums will often fail to catch it. A well-chosen CRC generator can detect burst errors with a success rate of over 99.99%.

6. Explain the Hamming Code.

Ans: Unlike Parity Checks or CRC, which only *detect* errors and rely on the sender to retransmit the corrupted data, Hamming Code goes a significant step further. It is an Error-Correcting Code (ECC). Developed by Richard Hamming in 1950, this technique can not only detect that a single-bit error has occurred during transmission but can also pinpoint exactly which bit flipped. This allows the receiving device to automatically flip it back and correct the error on the fly without ever requesting a retransmission.

Here is a breakdown of how Hamming Code works:

1. The Concept of Overlapping Redundancy

Hamming Code works by interspersing multiple parity bits into the actual data before transmission. Unlike a simple parity check that uses one bit for the whole block, Hamming Code uses multiple parity bits, each responsible for checking a specific, overlapping subset of the message.

By analyzing which specific parity checks fail at the receiver, a mathematical puzzle is solved to isolate the exact location of the corrupted bit.

2. Determining the Number of Parity Bits

Before we can place any parity bits, we need to mathematically determine how many are required for a given amount of data. This is done using the Hamming Rule (or Hamming inequality):

$2^r >= m + r + 1$
Where:

- r = number of redundant (parity) bits
- m = number of data bits

Example: If we want to send 4 bits of data (m = 4):

- If we try r = 2: $2^2 >= 4 + 2 + 1$ --> $4 >= 7$ (This is False, 2 bits are not enough).
- If we try r = 3: $2^3 >= 4 + 3 + 1$ --> $8 >= 8$ (This is True).

So, to protect 4 data bits, we must add 3 parity bits. The total message size sent over the network will be 7 bits (4 + 3). This specific configuration is famously known as the Hamming (7,4) code.

3. Positioning the Bits

Once we know how many bits, we have in total (7 in our example), we number the positions starting from 1 from left to right.

The rule for placement is based on powers of 2:

- Parity Bits are placed at positions that are powers of 2 (Positions 1, 2, 4, 8, 16, etc.). Let's call them P_1, P_2, P_4.
- Data Bits fill in the remaining gaps (Positions 3, 5, 6, 7, 9, 10, etc.). Let's call them D_3, D_5, D_6, D_7.

So, our 7-bit message structure looks like this:

[P_1] [P_2] [D_3] [P_4] [D_5] [D_6] [D_7]

4. Calculating the Parity Bits (The Sender)

Each parity bit checks a specific alternating pattern of bits, based on the binary representation of their position numbers. Assuming we are using Even Parity (the total number of 1s in the checked group must be even):

- P_1 (Position 1): Checks 1 bit, skips 1, checks 1, skips 1...
    - It checks positions: 1, 3, 5, 7, 9, 11...
- P_2 (Position 2): Checks 2 bits, skips 2, checks 2, skips 2...
    - It checks positions: 2, 3, 6, 7, 10, 11...
- P_4 (Position 4): Checks 4 bits, skips 4, checks 4...
    - It checks positions: 4, 5, 6, 7, 12, 13, 14, 15...

The sender looks at the data bits in each group and sets the respective Parity Bit to 0 or 1 to ensure the group has an even number of 1s.

5. Error Detection and Correction (The Receiver)

When the receiver gets the 7-bit message, it doesn't know if an error occurred. It recalculates the parity for each of the three groups to find out.

- It checks the group covered by P_1 (bits 1, 3, 5, 7). If the number of 1s is even, it writes down a 0 (Pass). If odd, it writes a 1 (Fail). Let's call this result C_1.
- It checks the group covered by P_2 (bits 2, 3, 6, 7), producing result C_2.
- It checks the group covered by P_4 (bits 4, 5, 6, 7), producing result C_4.

The receiver combines these three results into a binary number reading from highest to lowest (C_4, C_2, C_1). This is called the Syndrome Word.

The Magic of the Syndrome Word:

- If the Syndrome Word is 000: Every parity check passed. No single-bit error occurred.
- If the Syndrome Word is NOT zero: The decimal value of the Syndrome Word reveals the exact position number of the flipped bit.

Example of Correction:

Imagine the receiver calculating the groups and finds that C_4 failed (1); C_2 failed (1), but C_1 passed (0).

The Syndrome Word is 110.

If you convert binary number 110 to decimal, it equals 6.

The receiver instantly knows that the bit at Position 6 was corrupted by interference. It simply flips the bit at Position 6 (from a 0 to a 1, or vice versa), and the message is perfectly repaired!

7. Explain the Stop and Wait ARQ protocol.

Ans: The Stop and Wait ARQ (Automatic Repeat reQuest) is the simplest flow control and error control protocol used at the Data Link Layer.

It is designed to ensure that data frames are sent reliably from a sender to a receiver over a noisy channel, preventing the sender from overwhelming the receiver and recovering any frames that get lost or corrupted along the way.

As the name implies, the core rule of this protocol is: The sender transmits one frame, stops, and waits for an acknowledgment from the receiver before sending the next frame.

Here is a breakdown of how it operates, including how it handles various error scenarios.

1. The Basic Mechanism (Error-Free Transmission)

In a perfect scenario where no data is lost or corrupted, the Stop and Wait ARQ follows a very straightforward ping-pong sequence:

1. The Sender: Takes a data packet from the Network layer, encapsulates it into a frame (adding a header and a CRC for error detection), and transmits the frame (let's call it Frame 0) to the receiver.
2. The Wait: The sender starts a timer immediately after transmitting the frame. It then goes idle and waits.
3. The Receiver: Receives Frame 0, uses the CRC to verify it is uncorrupted, and passes the data up to its Network layer.
4. The Acknowledgment (ACK): The receiver sends a small, special control frame back to the sender called an Acknowledgment (ACK). This ACK explicitly tells the sender, "I received Frame 0 successfully; I am ready for the next one."
5. The Next Frame: The sender receives the ACK, stops its timer, and is now free to transmit the next frame (Frame 1).

Handling Errors

Because real-world networks are noisy, frames and ACKs can be delayed, corrupted, or completely lost. Stop and Wait ARQ uses three main tools to handle these issues: Timeouts, Sequence Numbers, and Retransmissions.

Scenario A: A Lost or Corrupted Data Frame

If the sender transmits a frame, but it is destroyed by interference or dropped by a faulty switch before it reaches the receiver:

1. The Sender: Transmits Frame 0 and starts its timer.
2. The Receiver: Never receives the frame, so it does nothing. (Or it receives a corrupted frame, detects the error using the CRC, discards it, and silently does nothing).
3. The Timeout: The sender timer eventually expires (times out) because it hasn't received an ACK.
4. The Retransmission: The sender assumes the frame was lost and simply retransmits an exact duplicate of Frame 0, starting the timer again.

Scenario B: A Lost Acknowledgment (ACK)

If the sender transmits a frame perfectly, but the receiver's ACK is lost on the way back:

1. The Sender: Transmits Frame 0 and starts its timer.
2. The Receiver: Receives Frame 0 perfectly, processes it, and sends an ACK back.
3. The Lost ACK: The ACK is destroyed in transit.
4. The Timeout & Retransmission: The sender's timer expires. Even though the receiver got the frame, the sender doesn't know that. The sender retransmits Frame 0.
5. The Duplicate Problem: The receiver gets Frame 0 *again*. If it blindly accepts it, it will process the same data twice.

The Solution: Sequence Numbers

To solve the duplicate problem, Stop and Wait ARQ adds a Sequence Number to the header of every frame. Because the protocol only ever sends one frame at a time, it only needs two sequence numbers: 0 and 1.

- The sender alternates: Frame 0, Frame 1, Frame 0, Frame 1...
- When the receiver gets the retransmitted Frame 0 in Scenario B, it looks at the sequence number, realizes it already successfully processed as a Frame 0 recently, and discards the duplicate. It then resends an ACK to tell the sender to move on.

Scenario C: A Delayed Acknowledgment

Sometimes, an ACK isn't lost, just severely delayed due to network congestion.

1. The Sender: Transmits Frame 0, starts timer.
2. The Receiver: Receives Frame 0, sends ACK 1 (meaning "ready Frame 1").
3. The Delay: ACK 1 gets stuck in traffic. The sender's timer expires.
4. The Retransmission: The sender retransmits Frame 0.
5. The Receiver: Gets the duplicate Frame 0, discards it, and sends another ACK 1.
6. The Confusion: The original delayed ACK 1 finally arrives at the sender. The sender thinks, "Great! I'll send Frame 1." It sends Frame 1. Right after, the second ACK 1 arrives.

The Solution: ACK Sequence Numbers

To prevent the sender from getting confused by delayed ACKs, the ACKs must also have sequence numbers. The receiver sends an ACK numbered with the sequence number of the *next frame it expects to receive*.

- If it receives Frame 0, it sends ACK 1.
- If it receives Frame 1, it sends ACK 0.

If the sender receives a delayed duplicate ACK, it can simply ignore it based on the sequence number.

Limitations of Stop and Wait

While simple to implement, Stop and Wait ARQ is incredibly inefficient over long distances. If the physical link is a satellite connection with high latency, the sender will spend the vast majority of its time sitting idle, waiting for ACKs, rather than actually transmitting data.

8. Explain Go Back N ARQ protocol.

Ans: The Go-Back-N ARQ (Automatic Repeat reQuest) protocol is a specific type of Sliding Window protocol used at the Data Link Layer to handle flow control and error control.

It was designed to directly solve the major inefficiency of the Stop-and-Wait ARQ protocol. Instead of sending one frame and halting all transmission until an acknowledgment (ACK) is received, Go-Back-N allows the sender to transmit multiple frames consecutively before requiring an ACK.

This continuous transmission keeps the network link busy, dramatically improving throughput, especially on connections with high latency (like satellite links).

Here is a detailed breakdown of how Go-Back-N ARQ operates:

1. The Concept of the "Sliding Window"

The core mechanism of Go-Back-N is the concept of a "window." This window represents the maximum number of unacknowledged frames the sender is allowed to have in transit at any given time.

- Window Size (N): The sender and receiver agree on a window size, let's call it N. For example, if N=4, the sender can transmit up to 4 frames back-to-back without waiting for a single ACK.
- Sequence Numbers: To keep track of these multiple frames, each frame header must contain a sequence number. If the window size is N, the sequence numbers typically range from 0 to 2^m - 1 (where m is the number of bits used for the sequence number in the header). For simplicity in our examples, let's assume sequence numbers 0, 1, 2, 3, 4...
- The "Sliding" Action: As the sender receives ACKs for the frames it has sent, the window "slides" forward, allowing the sender to transmit new frames.

2. Normal Operation (Error-Free)

Let's assume a window size of N=4.

1. Continuous Transmission: The sender transmits Frame 0, Frame 1, Frame 2, and Frame 3 consecutively.
2. The Window is Full: The sender has now sent 4 unacknowledged frames. It must stop transmitting new data and wait.
3. Cumulative Acknowledgments: The receiver processes the frames as they arrive *in order*. Instead of sending an individual ACK for every single frame, Go-Back-N often uses cumulative ACKs.
    a. If the receiver successfully gets Frame 0 and Frame 1, it might send back ACK 2.

    b. This ACK 2 means: *"I have successfully received all frames up to number 1, and I am now expecting Frame 2."*
4. The Window Slides: When the sender receives ACK 2, it knows Frames 0 and 1 are safe. Its window slides forward with two slots. It is now allowed to transmit Frame 4 and Frame 5.

3. Handling Errors (The "Go-Back-N" Rule)

The name "Go-Back-N" comes exactly from how this protocol handles a lost or corrupted frame.

The Golden Rule of the Receiver: The receiver in Go-Back-N is very rigid. It will only accept frames in the exact correct sequence. It has no memory or buffer for frames that arrive out of order.

Let's look at what happens when an error occurs (assuming N=4):

Scenario: A Lost Data Frame

1. Transmission: The sender transmits Frame 0, Frame 1, Frame 2, and Frame 3.
2. The Error: Frame 1 is lost in transit due to interference.
3. The Receiver's Reaction: It receives Frame 0 perfectly and passes it up.
    a. It *expects* Frame 1 next.
    b. Instead, Frame 2 arrives. Because it is out of sequence, the receiver discards Frame 2.
    c. Frame 3 arrives. Because it is still expecting Frame 1, it discards Frame 3.
    d. For every out-of-order frame it receives, the receiver typically sends an ACK for the last successfully received frame (in this case, it might repeatedly send ACK 1).
4. The Sender's Timeout: The sender has a timer running for the oldest unacknowledged frame (Frame 1). Eventually, this timer expires.
5. The "Go-Back-N" Retransmission: This is the defining characteristic of the protocol. When the timer for Frame 1 expires, the sender realizes there is a problem. It must go back to the point of failure (Frame 1) and retransmit every single frame it sent after that point, even if some of them (like Frame 2 and 3) might have actually reached the receiver perfectly.
    a. The sender retransmits: Frame 1, Frame 2, and Frame 3.

Scenario: A Lost Acknowledgment

Because Go-Back-N uses cumulative ACKs, a lost ACK is often automatically corrected without any retransmissions.

1. Transmission: The sender sends Frame 0 and Frame 1.
2. The Receiver: Gets both frames perfectly. It sends ACK 1 (expecting Frame 1) and then sends ACK 2 (expecting Frame 2).
3. The Error: ACK 1 gets lost in the network.
4. The Recovery: Before the sender timer for Frame 0 expires, ACK 2 arrives.
5. The Sender's Logic: The sender receives ACK 2. The cumulative nature of the ACK tells the sender, *"The receiver is expecting Frame 2, which means it must have successfully received Frame 0 AND Frame 1."* The sender slides its window forward, and no retransmissions are needed!

Advantages and Disadvantages

Advantages:

- High Efficiency: Keeps the network pipe full, making it vastly superior to Stop-and-Wait on high-latency connections.
- Simple Receiver: The receiver's logic is very simple. It doesn't need complex memory buffers to store out-of-order frames; it just discards them.

Disadvantages:

- Wasted Bandwidth on Errors: If the network is noisy and drops frames frequently, Go-Back-N is very inefficient. Retransmitting a whole sequence of N frames just because one was lost wastes a lot of bandwidth.

9. Explain the Selective Repeat ARQ protocol.

Ans: The Selective Repeat ARQ protocol is the most advanced and efficient flow control and error control mechanism in the Data Link Layer's Sliding Window protocol family.

It was designed to fix the major flaw of the Go-Back-N protocol: wasted bandwidth. In Go-Back-N, if a single frame is lost, the sender must retransmit that frame *and every single frame sent after it*, even if those subsequent frames arrived perfectly.

Selective Repeat solves this by adding complexity (memory and logic) to the receiver. As the name suggests, if an error occurs, the sender only selectively repeats the specific frame that was lost or corrupted, rather than the whole sequence.

Here is a breakdown of how Selective Repeat ARQ works to maximize network efficiency:

1. The Core Differences from Go-Back-N

To achieve this efficiency, Selective Repeat requires three major changes compared to Go-Back-N:

- Independent Timers: Instead of one timer for the oldest unacknowledged frame, the sender must maintain a separate, independent timer for *every single frame* currently in its sliding window.
- Receiver Window (Memory Buffer): In Go-Back-N, the receiver only has a window size of 1 (it only accepts the specific frame it expects next). In Selective Repeat, the receiver also has a sliding window of size N. This allows the receiver to accept and store frames that arrive *out of sequence*.
- Individual Acknowledgments: Selective Repeat moves away from cumulative ACKs. The receiver must send an individual ACK for each specific frame it successfully receives.

2. The Window Size Rule

For Selective Repeat to work correctly and prevent the receiver from confusing a new frame with a retransmitted old frame, there is a strict mathematical rule for the window size (N).

The sender window size (N) and the receiver window size (N) must be equal, and their combined size cannot exceed the total number of available sequence numbers.

If we are using m bits for sequence numbers:

Total Sequence Numbers = 2^m
Sender Window (N) + Receiver Window (N) <= 2^m
Therefore: Window Size (N) <= 2^m-1

Example: If we use 3 bits for sequence numbers (0 through 7), the maximum window size for Selective Repeat is 2^3-1 = 2^2 = 4. (In Go-Back-N, the window size could be up to 7).

3. Normal Operation (Error-Free)

Let's assume a window size of N=4 and sequence numbers 0, 1, 2, 3.

1. Transmission: The sender transmits Frame 0, Frame 1, Frame 2, and Frame 3 consecutively, starting an independent timer for each.
2. Reception: The receiver gets all four frames perfectly.
3. Individual ACKs: The receiver sends back ACK 0, ACK 1, ACK 2, and ACK 3.
4. Sliding the Window: As the sender receives each specific ACK, it stops that frame timer and slides its window forward, allowing it to send new frames (Frame 4, etc.).

4. Handling Errors (The "Selective Repeat" Action)

This is where the protocol shines. Let's look at the scenario where a data frame is lost (assuming N=4):

1. Transmission: The sender transmits Frame 0, Frame 1, Frame 2, and Frame 3.
2. The Error: Frame 1 is lost in transit due to interference.
3. The Receiver's Reaction (Buffering):
    a. It receives Frame 0 perfectly, passes it up to the Network layer, and sends ACK 0. Its window slides forward.
    b. It expects Frame 1, but Frame 2 arrives instead.
    c. *Unlike Go-Back-N, which discards it*, the Selective Repeat receiver stores Frame 2 in its memory buffer and sends ACK 2.
    d. Frame 3 arrives. The receiver stores Frame 3 in its buffer and sends ACK 3.
4. The Sender's Logic: The sender receives ACK 0, ACK 2, and ACK 3. It stops the timers for those specific frames. However, the timer for Frame 1 is still running.
5. The Timeout & Selective Retransmission: The independent timer for Frame 1 eventually expires. The sender realizes that Frame 1 was lost. It selectively retransmits only to Frame 1. It does *not* retransmit Frames 2 and 3 because it already received ACKs for them.
6. The Receiver Completes the Sequence: The receiver finally gets the retransmitted Frame 1. It sends ACK 1. Because it already has Frames 2 and 3 safely stored in its buffer, it can now pass Frame 1, Frame 2, and Frame 3 up to the Network layer in the correct consecutive order. Its window slides forward significantly.

Handling a Lost Acknowledgment

If an ACK is lost, the sender timer for that specific frame will expire, and it will selectively retransmit that frame.

Because the receiver already has that frame safely stored (and likely already passed it to the Network layer), when the duplicate frame arrives, the receiver simply discards the duplicate data but resends the ACK to ensure the sender knows it arrived safely.

Advantages and Disadvantages

Advantages:

- Maximum Efficiency: It wastes almost zero bandwidth on retransmissions. It is the best protocol for noisy, high-error-rate links (like wireless networks) or high-latency links (like satellite communication).

Disadvantages:

- High Complexity: It requires significantly more processing logic on both ends.
- Memory Requirements: The receiver needs dedicated memory buffers to store out-of-order frames, which increases the hardware cost and complexity of the receiving device.

10. What are multiple access protocols?

Ans: When a network link is shared by multiple devices—such as several computers connected to a single Ethernet hub or dozens of phones connected to the same Wi-Fi router—it is known as a broadcast link or a multipoint connection.

Because they share the exact same physical medium (the cable or the radio frequency), a major problem arises: if two or more devices try to transmit data at the exact same time, their electrical or radio signals will crash into each other. This is called a collision, and it corrupts the data from all transmitting devices.

Multiple Access Protocols are the set of rules that dictate how these devices share the channel and take turns transmitting to prevent or recover from these collisions. These protocols operate within the Medium Access Control (MAC) sublayer of the Data Link Layer.

You can think of these protocols, like the rules of conversation at a busy dinner party.

There are three main categories of Multiple Access Protocols:

1. Random Access Protocols (Contention-Based)

In this approach, no single device has special priority or control over the others. Any device can try to transmit data whenever it wants. Because everyone is "contending" for the channel, collisions are expected, and these protocols focus heavily on how to handle them.

- ALOHA: The earliest and simplest method.
    - Pure ALOHA: A device transmits whenever it has data. If it doesn't receive an acknowledgment (meaning a collision occurred), it waits for a random amount of time and tries again. It has a very high collision rate.
    - Slotted ALOHA: Improves Pure ALOHA by dividing time into strict "slots." Devices can only begin transmitting at the exact start of a slot, which significantly reduces the chance of frames partially overlapping.
- CSMA (Carrier Sense Multiple Access): This adds a "listen before you talk" rule. Before a device transmits, its "senses" the wire or frequency. If it hears someone else transmitting, it waits.
- CSMA/CD (Collision Detection): Used heavily in traditional wired Ethernet. A device listens before transmitting, but it also listens *while* transmitting. If it detects a collision, it immediately stops, sends out a "jam signal" to warn everyone else, waits for a random backoff time, and tries again.
- CSMA/CA (Collision Avoidance): Used in wireless networks (Wi-Fi). Because a wireless device cannot easily listen and transmit at the same time to detect collisions, it focuses on *avoiding* them entirely. It uses complex timers and sometimes requests permission (RTS/CTS - Request to Send / Clear to Send) before transmitting actual data.

2. Controlled Access Protocols

In these protocols, the "dinner party" has a moderator. Devices cannot just transmit whenever they want; they must be explicitly authorized or wait for their designated turn. This eliminates collisions entirely but adds overhead.

- Reservation: Time is divided into phases. Before data can be sent, devices must first send a tiny message during a "reservation phase" to claim a time slot for their actual data transmission.
- Polling: One device is designated as the "Primary" (like a mainframe), and all others are "Secondaries" (like dumb terminals). The Primary explicitly asks (polls) each Secondary one by one, "Do you have data to send?" A Secondary can only speak when spoken to.
- Token Passing: A special, tiny control frame called a "token" continuously circulates around the network from device to device (often used in Ring topologies). A device can only transmit data if it is currently holding the token. Once it finishes sending, it passes the token to the next neighbor.

3. Channelization Protocols (Multiplexing)

Instead of taking turns in time, Channelization divides the actual physical capacity of the link into separate, dedicated pathways for each device.

- FDMA (Frequency Division Multiple Access): The total available frequency bandwidth is sliced into smaller, distinct frequency bands. Each device gets its own dedicated band, like separate lanes on a highway. (Used in classic analog television and radio).
- TDMA (Time Division Multiple Access): The entire frequency of bandwidth is used, but devices take strict, rapidly repeating turns. Device A gets a millisecond, then Device B, then Device C, then back to A. (Used in 2G cellular networks).
- CDMA (Code Division Multiple Access): All devices transmit at the exact same time, using the exact same frequency band. However, each device mathematically encodes its data using a unique,

orthogonal mathematical code. The receiver can filter out all other signals such as "background noise" and extract only the signal with the matching code. (Used in 3G cellular networks.

11. Explain the CSMA/CD protocol.

Ans: CSMA/CD (Carrier Sense Multiple Access with Collision Detection) is the foundational protocol that governed how devices communicated on early wired Ethernet networks. It sits within the Medium Access Control (MAC) sublayer of the Data Link Layer.

To understand it, imagine a group of people sitting in a dark room trying to have a conversation over a single telephone line.

Here is a breakdown of what the acronym means and how the protocol operates step-by-step:

1. The Three Pillars of CSMA/CD

- CS (Carrier Sense) - "Listen Before You Talk": Before a network interface card (NIC) sends any data onto the cable, it first "listens" to the wire to see if there is a carrier signal (meaning someone else is already transmitting).
- MA (Multiple Access) - "Shared Medium": This simply means that multiple computers are connected to the exact same physical wire (like a bus topology or through a basic network hub).
- CD (Collision Detection) - "Listen While You Talk": This is the defining feature. Even while a device is transmitting its own data, it continues to monitor the voltage levels on the wire. If the voltage suddenly spikes beyond a normal level, it knows its transmitted signal just crashed into another device's signal. This is a collision.

2. The Step-by-Step Operation

When a computer has a data frame ready to send over the network, it follows this strict sequence:

1. Sense the Channel: The device checks the physical wire.
2. If Busy: If it hears data already on the wire, it waits until the wire is quiet. (How long it waits depends on a specific "persistence strategy").
3. If Idle: If the wire is completely quiet, the device begins transmitting its frame bit by bit.
4. Monitor for Collisions: As it transmits, it actively listens.
    a. Success: If it finishes sending the entire frame and never detects a voltage spike, the transmission is successful.
    b. Collision Detected: If it detects a collision before it finishes sending the frame, it immediately moves to the recovery phase.

3. Collision Recovery: The Jam Signal and Backoff

When a collision happens, the data from both transmitting devices is completely corrupted. CSMA/CD handles this efficiently to prevent the network from locking up.

Step A: The Jam Signal

The moment a device detects a collision, it stops sending its actual data frame and instead transmits a brief, 32-bit Jam Signal (typically a repeating pattern of 1s and 0s). This ensures that every other device on the network clearly recognizes that a collision just occurred and that they should discard any fragments of data they just received.

Step B: The Backoff Algorithm

If both computers involved in the collision realize what happened, stop, and immediately try to retransmit at the exact same time, they will just collide again. To solve this, CSMA/CD uses the Truncated Binary Exponential Backoff algorithm.

This algorithm forces the devices to wait a random amount of time before trying again. Crucially, the window of random time *doubles* after every consecutive collision.

Here is how the math works:

- Let c be the number of consecutive collisions a specific frame has experienced.
- The device chooses a random number (R) between 0 and 2^c - 1.
- The device waits for R * (slot of time) before checking if the wire is idle again to retransmit.

Example of Exponential Backoff:

- 1st Collision (c=1): The device picks a random number between 0 and 2^1 - 1 (which is 0 to 1). It waits either 0 or 1 time slot.
- 2nd Collision (c=2): If it tries again and collides *again*, it picks a random number between 0 and 2^2 - 1 (which is 0 to 3). It waits for 0, 1, 2, or 3 time slots.
- 3rd Collision (c=3): It picks a random number between 0 and 7.

By exponentially increasing the range of random numbers, the protocol rapidly spreads the retransmission attempts, drastically reducing the chance of a repeat collision, even on a highly congested network.

(Note: If a single frame experiences 16 consecutive collisions, the network interface card assumes the network is critically broken, gives up, and drops the frame entirely, reporting an error to the upper layers.

12. Explain the MAC address.

Ans: Within the Data Link Layer, specifically in the Medium Access Control (MAC) sublayer, MAC

addressing is the system used to uniquely identify every single piece of network hardware on a local network segment.

Unlike IP addresses (which are logical and can change depending on what network you connect to), a MAC address is a physical address. It is permanently burned into the hardware of your Network Interface Card (NIC) at the factory by the manufacturer.

Here is a detailed breakdown of how MAC addressing works and its structure:

1. The Purpose of a MAC Address

When data is traveling across the internet, it uses IP addresses to navigate from router to router (the Network Layer). However, once the data arrives at the final local network (like your home Wi-Fi or a corporate office network), IP addresses are no longer enough to get the data to the exact right computer.

The Data Link Layer takes the data packet, puts it inside a frame, and slaps the Destination MAC Address and Source MAC Address on the outside. The local network switches read these MAC addresses to physically deliver the frame to the correct hardware port.

Think of it like delivering mail:

- The IP Address is the zip code and street name. It gets the letter to the right apartment building.
- The MAC Address is the specific name and apartment number in the mailbox. It guarantees the letter gets to the exact right person in that building.

2. The Structure of a MAC Address

A standard MAC address is exactly 48 bits long.

Because looking at 48 continuous 1s and 0s is nearly impossible for humans to read, MAC addresses are almost always displayed as a series of 12 Hexadecimal digits (where each hex digit represents 4 bits).

These 12 digits are typically grouped in pairs, separated by colons (:), hyphens (-), or sometimes periods (.).

Example Formats:

- 00:1A:2B:3C:4D:5E
- 00-1A-2B-3C-4D-5E
- 001A.2B3C.4D5E

3. How a MAC Address is Assigned (OUI and UAA)

The 48 bits of a MAC address are strictly divided into two equal halves to ensure that no two network cards in the world ever accidentally have the same address.

The First Half (24 bits): The OUI

The first 6 hexadecimal digits are called the Organizationally Unique Identifier (OUI).

- This half acts as a manufacturer ID.
- The IEEE (Institute of Electrical and Electronics Engineers) manages and assigns these OUIs to companies. For example, Apple, Cisco, Intel, and Dell all have specific OUIs assigned to them. If you look up the first half of your device's MAC address, you can tell who made the network chip.

The Second Half (24 bits): The UAA

The last 6 hexadecimal digits are called the Universally Administered Address (UAA) or the Vendor-Assigned portion.

- This half acts as the serial number.
- The manufacturer is responsible for ensuring that every network card they produce with their OUI has a completely unique UAA.

4. Types of MAC Addresses

When a frame is sent on a local network, the destination MAC address determines who should process the frame. There are three types:

- Unicast: The destination is a single, specific MAC address. The frame is meant for one specific device. (Example: 00:1A:22B:3C: 4D:5E.
- Multicast: The frame is meant for a specific group of devices on the network. Multicast MAC addresses always have the least significant bit of the first byte set to 1. (Example: 01:00:5E:7F: FF:FF).
- Broadcast: The frame is meant for *every single device* on the local network segment. A broadcast MAC address is always all 1s in binary, which translates to all Fs in hexadecimal. Every device that receives this frame will be processed. (Address: FF: FF:FF: FF: FF: FF).

13. Explain Ethernet technology.

Ans: Ethernet is the absolute foundation of modern wired local area networks (LANs). When you plug a physical cable into your computer or a network switch to get internet access, you are almost certainly using Ethernet technology.

It operates primarily at the lower two layers of the OSI model: the Physical Layer (Layer 1) and the Data Link Layer (Layer 2).

Here is a comprehensive breakdown of how Ethernet technology works, from its physical cables to how it organizes data into frames.

1. The Physical Layer of Ethernet

The physical layer defines the actual cables, connectors, and electrical signals used to transmit data bits. Ethernet has evolved dramatically over the decades, supporting increasingly faster speeds over different physical media.

A. Twisted Pair Cables (Base-T):

This is the most common type of Ethernet cabling used in homes and offices. It uses copper wires twisted together to cancel electromagnetic interference.

- Connectors: They use standard RJ-45 connectors (the familiar plastic plug with 8 pins).
- Categories: The cables are rated by "Categories" (Cat).
    - Cat5e: Supports up to 1 Gbps (Gigabit Ethernet) over short distances.
    - Cat6 / Cat6a: Supports up to 10 Gbps.
    - Cat8: Supports up to 40 Gbps over very short distances (usually in data centers).

B. Fiber Optic Cables (Base-X / Base-R):

Instead of electrical signals over copper, fiber optics use pulses of light transmitted through glass or plastic strands.

- Use Cases: Used for long-distance connections, backbone links between network switches, and extremely high-speed connections (10 Gbps, 40 Gbps, 100 Gbps, and beyond).
- Advantages: Immune to electrical interference and can carry data much further than copper without signal degradation.

2. The Data Link Layer: The Ethernet Frame

When data is passed down from the upper layers (like an IP packet from the Network layer), Ethernet encapsulates it into a structured Ethernet Frame.

The most common and widely used frame format today is Ethernet II (also known as DIX Ethernet).

Here is the breakdown of an Ethernet II frame, piece by piece:

1. Preamble (7 Bytes): A pattern of alternating 1s and 0s (`10101010`). This acts as a "wake-up call" to the receiving device, allowing its circuitry to synchronize its clock with the incoming signal.
2. Start Frame Delimiter (SFD) (1 Byte): A specific pattern (`10101011`) that signals the exact end of the synchronization phase and tells the receiver, "The actual frame data starts *now*."
3. Destination MAC Address (6 Bytes): The physical address of the device intended to receive the frame. (This can be a unicast, multicast, or broadcast address).
4. Source MAC Address (6 Bytes): The physical address of the device sending the frame.
5. EtherType / Length (2 Bytes): In Ethernet II, this field is almost always used as an EtherType. It tells the receiving Data Link layer which protocol is sitting inside the payload (e.g., `0x0800` means it contains an IPv4 packet; `0x86DD` means it contains an IPv6 packet).

6. Payload / Data (46 to 1500 Bytes): This is the actual "cargo" being carried (usually an IP packet).
    a. Minimum Size: The payload *must* be at least 46 bytes. If the actual data is smaller, "padding" (junk data) is added to reach 46 bytes. This minimum size was historically required for CSMA/CD to detect collisions properly on long cables.
    b. Maximum Size (MTU): The standard Maximum Transmission Unit is 1500 bytes. (Some specialized networks support "Jumbo Frames" up to 9000 bytes).
7. Frame Check Sequence (FCS) (4 Bytes): This is the trailer. It contains a 32-bit Cyclic Redundancy Check (CRC-32) value calculated over the entire frame (excluding the Preamble and SFD). The receiver recalculates this to ensure no bits were corrupted during transit.

3. How Ethernet Devices Communicate (MAC and Switches)

In the early days of Ethernet, devices were connected using a shared coaxial cable (a bus topology) or through a basic network hub. This meant every device heard every frame sent on the network, leading to collisions managed by the CSMA/CD protocol (which we discussed earlier).

Modern Ethernet Uses Switches:

Today, true shared-medium Ethernet and hubs are obsolete. Modern Ethernet networks use Network Switches and a star topology.

- Dedicated Links: Every device has its own dedicated twisted-pair cable running directly to a port on the switch.
- Full-Duplex: Modern connections are "full-duplex," meaning a device can send and receive data simultaneously on separate wires within the cable.
- No Collisions: Because each device has a dedicated, full-duplex link to the switch, collisions are physically impossible. Therefore, the CSMA/CD collision detection mechanism is effectively turned off in modern switched Ethernet networks.

The Role of the Switch:

An Ethernet switch is a highly intelligent Layer 2 device.

1. Learning: When a frame arrives on a port, the switch looks at the Source MAC Address and learns which physical port that specific device is connected to. It builds a "MAC Address Table."
2. Forwarding: It then looks at the Destination MAC Address. It checks its table, finds the correct port for that destination, and forwards the frame *only* out of that specific port. It does not broadcast the frame to everyone (unless it's an explicit broadcast address).

14. In Stop-and-Wait ARQ, if the frame transmission time is 2 ms and propagation delay is 10 ms, calculate channel utilization.

Ans: To calculate the channel utilization (also known as efficiency) for the Stop-and-Wait ARQ protocol, we use the following formula:

$U = T_t/(T_t + 2T_p)$
Where:

- U = Channel Utilization (Efficiency)
- T_t = Transmission time of the data frame
- T_p = Propagation delay (time taken for the frame to travel from sender to receiver)

Note: This standard formula assumes that the transmission time of the Acknowledgment (ACK) frame is negligible, and processing delay is zero.

1. Identify the Given Values

- Transmission Time (T_t) = 2 ms
- Propagation Delay (T_p) = 10 ms

2. Substitute the Values into the Formula

$$U = 2/(2 + 2(10))$$

3. Perform the Calculation

$U = 2/(2 + 20)$
$U = 2/22$
$U = 1/11$
U is approx. 0.0909

4. Convert to Percentage

To express this as a percentage, multiply by 100: 0.0909 * 100 = 9.09%

Answer: The channel utilization for this Stop-and-Wait ARQ connection is 9.09%.

15. Explain the difference between logical address and physical address.

Ans:

Feature	Logical Address (IP Address)	Physical Address (MAC Address)
Definition	An address generated by the network software to uniquely identify a device logically across networks.	An actual hardware address burned into the Network Interface Card (NIC) to identify a device locally.

OSI Model Layer	Operates at the Network Layer (Layer 3).	Operates at the Data Link Layer (Layer 2).
Address Length	32 bits (IPv4) or 128 bits (IPv6).	48 bits.
Representation	Dotted decimal (e.g., 192.168.1.1) or hexadecimal for IPv6.	Hexadecimal separated by colons or hyphens (e.g., 00: 1A:2B: 3C:4D:5E).
Changeability	Dynamic; it changes when the device moves and connects to a completely different network.	Static; it is permanent and remains the same regardless of where the device connects.
Assignment	Assigned by a network administrator or dynamically via DHCP (Dynamic Host Configuration Protocol).	Assigned by the hardware manufacturer at the factory.
Visibility/Scope	Used for global routing across the internet or between different subnets.	Used only for local, hop-to-hop delivery within the exact same local area network (LAN).